Using MATLAB to analyze rod balancing data

Aim

Programming is an essential skill for a scientist. It allows you to explore and analyses your data unconstrained by software packages. When you program your own data analysis you know and understand exactly how your results are produced. You can also use programming to build and run experiments exactly as you want, without being dependent on someone else to create it for you. Finally, programming is a good exercise for a scientist to develop logical, empirical, and critical thinking.

While there are several programming languages available (C++, Python, fortan etc), MATLAB is probably the easiest and most productive computing environment for engineers and scientists. In neuroscience, MATLAB is used almost universally. This is useful for individual scientists who might want to transition between research groups, and good for science generally, since it allows easy sharing of data and data analysis techniques.

This session has three parts. First, we will go over the basics of using MATLAB. Second, we will develop an analysis 'pipeline' using simulated data. Finally, we will load and process the data we collected in the first session to test our hypotheses. Work in your group. If you already have some programming skills, support your colleagues. If you get stuck, ask for help from your colleagues first. Each group should produce a short report that describes the experiment, the analysis, the results (with figures) and any conclusions that can be drawn.

Part 1: Using MATLAB

You need a licence to use MATLAB. University computers have access to the university licence server. If you have downloaded from the university MATLAB your computer needs to be connected to the university server.

https://library.maastrichtuniversity.nl/services/remote-access/

https://athenadesktop.maastrichtuniversity.nl/

Follow the 'Getting Started with MATLAB' on the MATLAB website.
https://nl.mathworks.com/help/matlab/getting-started-with-matlab.html
There are 9 sections. By the end you should be able to:

- Make and use variable, arrays, matricies and text strings
- Use functions
- Make and annotate plots (graphs)

Part 2a: Some fake data and relevant functions

In this part we are going to create some fake data to practice the analysis we will use on our real data. It is often helpful to do this before even starting to collect data in a real experiment. It gives you a chance to think about what kind of data to expect and what you plan to do with it.

Start by creating a new script (blank document). It is a good idea to work in a script rather than in the command window so you can edit and save your work. Add each command as a new line in your script. Run the script using the green Run arrow at the top of the page. Save your script with the title 'FakeData' (we will use this script again in the 4th session).

It is good practice to start a script by clearing the workspace. This saves memory and makes sure there are no variables you don't want. Also, close any figures that might be open and clear the command window (using clc).

```
clear all, close all, clc
```

1. Start simple with a single array of normally distributed random numbers using the function randn. Check how big the array is.

```
clear all, close all, clc
data1=randn(1,1000);
size(data1)
```

in the command window you should see that 'data1' is a 1 by 1000 matrix. Calculate some descriptive statistics, the mean and standard deviation.

```
clear all, close all, clc
data1=randn(1,1000);
size(data1)
mean(data1)
std(data1)
```

you should find that the mean is about 0 and the standard deviation is about 1.

```
Command Window

ans =

     1     1000


ans =

   -0.0326


ans =

    0.9990

fx >>
```

2. Make a histogram of the data to see what the distribution looks like.

```
clear all, close all, clc
data1=randn(1,1000);
size(data1)
mean(data1)
std(data1)
hist(data1)
```

The function hist automatically creates a figure with a 10 bin histogram. We can do better than that! hist can also give the counts in each bin and the bin centers as outputs. You can also set the

number of bins you want (or the exact bin centers). When you ask for outputs, the function does not make a plot by itself. You need to do this yourself.

```
[n,xout]=hist(data1,25);
bar(xout,n)
```

It is often useful to represent the height of a histogram in terms of proportions not actual counts. It can also be helpful to plot the distribution as a line rather than as bars.

```
[n,xout]=hist(data1,25);
plot(xout,n/sum(n))
```

3. Add a line to show the mean of the distribution. Use the function plot to make a vertical line. The X values of the line will be the mean of the data. The Y values will be 0 and the maximum of the histogram. Use square brackets to make an array of two numbers for the X and Y values.

```
plot([mean(data) mean(data)],[0 max(n/sum(n))])
```

This line of code might look complicated. But look at each part and you will see that it is not. We are plotting two values for X (both the same) surrounded by square brackets, and two values for Y (0 and the maximum of n divided by the sum of n).

If you get an error, check that your brackets are all correct and that there is no space between a function and its input mean(data) not mean (data). Are you using an input variable that exists?

If you run your script you will get a figure with only the vertical line plotted. MATLAB automatically deletes old parts of a figure when it plots a new part. Use the function hold to tell it to keep the lines you are plotting. Put it after the first bit of plotting.

```
clear all, close all, clc
data1=randn(1,1000);
size(data1)
mean(data1)
std(data1)
[n,xout]=hist(data1,25);
plot(xout,n/sum(n)), hold on
plot([mean(data) mean(data)],[0 max(n/sum(n))])
```

now you should see both parts of the plot.

4. Annotate your plot. A graph should always have axis labels and usually a title and legend.

```
clear all, close all, clc
data1=randn(1,1000);
size(data1)
mean(data1)
std(data1)
[n,xout]=hist(data1,25);
plot(xout,n/sum(n)), hold on
plot([mean(data1) mean(data1)],[0 max(n/sum(n))])
title('data distribution')
xlabel('value')
ylabel('proportion')
legend([{'data'},{'mean'}])
```

Notice that you are using strings as inputs to the functions title, xlabel, ylabel and legend. They should appear purple in your script.

5. Test whether the distribution is different from a particular mean value using a one sample t-test. The function t-test has two inputs, the data and the value you want to test against, and several outputs, H whether the null hypothesis is correct is not, P the p-value, CI the confidence intervals, and STATS that contains the t-value and degrees of freedom.

```
[H, P, CI, STATS]=ttest(data1,0)
```

When you report the results of a t-test you report write t(degrees of freedom)=t-value, p= P value. You can get mahlab to make a string with this information. You need to use the function num2str to convert numbers to strings.

```
test=['t(' num2str(STATS.df) ')=' num2str(STATS.tstat) ' ,p=' num2str(P)]
```

Remember to make sure that there is a space between each part of the string but not between the function num2str and its input
't(' num2str(STATS.df) ')=' not 't('num2str(STATS.df)')='

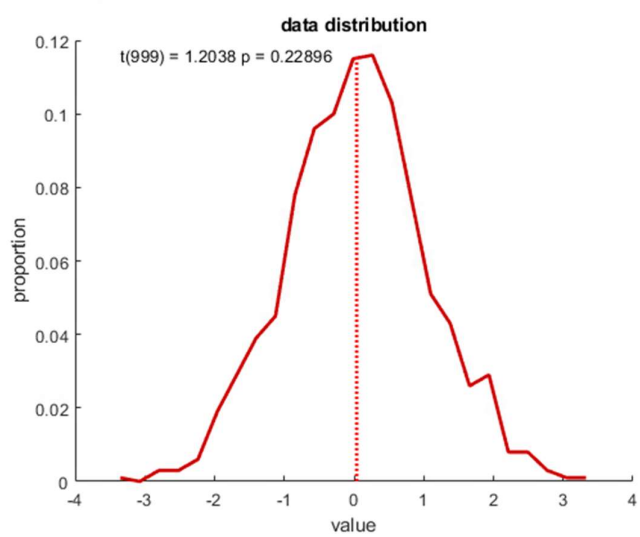
```
Command Window
>> test=['t(' num2str(STATS.df) ')=' num2str(STATS.tstat) ' ,p=' num2str(P)]

test =

    't(999)=0.1229 ,p=0.90221'

>>
```

6. Now you have a useful script for plotting the distribution of some data. Try changing the data.
   1) Try use rand instead of randn.
   2) Try having 100000 data points instead of 1000.
   3) Try multiplying data by a value, or adding a value to it e.g.

data1=randn(1,1000)+5

At the end of part 2a my script and figure look like this. I have added some comments (in green, using the % sign) so that I know what I am doing in each part. I have also added some blank lines to break the script up. This also helps reading the script.

```
%%% part 2a ones sample ttest
% make some data
data1=randn(1,1000);
size(data1),mean(data1),std(data1)

% plot a histogram
[n,xout]=hist(data1,25);
plot(xout,n/length(data1),'-','linewidth',2,'color',[0.8 0 0]); hold on
plot([mean(data1) mean(data1)],[0 max(n/sum(n))],'r:','linewidth',2)

%anotation
title('data distribution'); xlabel('value');ylabel('proportion');
box off; %switch off the box around the figure
set(gcf,'color','w') % set the color of the figure to white

%statistical test
[H,P,CI,STATS] = ttest(data1);
test=['t(' num2str(STATS.df) ') = ' num2str(STATS.tstat) ' p = ' num2str(P) ];
text(xout(1),max(n/sum(n)),test)
```



Part 2b: Some *more* fake data and relevant functions

In many experiments you will compare two datasets.

1. Create a second array of data called data2 and copy the script you already have, to plot the distribution. Make data2 look different from your first set of data by adding a value to it. You script might look like this.

```
%%%%%%%%%%%%%%%%%% Part 2 (part 3 of the worksheet) %%%%%%%%%%%%%%%

clear all, close all, clc

% make some data and get descriptives
data1 = randn(1,1000);
data2 = randn(1,1000)+1;

% plot a histogram in the first subplot
subplot(2,2,1)

% data1
[n, xout] = hist(data1, 25);
plot(xout, n/sum(n)), hold on
plot([mean(data1) mean(data1)], [0 max(n/sum(n))])

% data2
[n, xout] = hist(data2, 25);
plot(xout, n/sum(n)), hold on
plot([mean(data2) mean(data2)], [0 max(n/sum(n))])

% annotation
title('data distribution')
xlabel('value')
ylabel('proportion')
legend([{'data1'}, {'data2'}, {'mean1'}, {'mean2'}])
box off % switch off the box around the figure
set(gcf, 'color', 'w') % set background colour of figure to white
```

2. Summarize the data in a bar plot of means in the second subplot

```
% plot a bar plot of means in the second subplot
subplot(2,2,2)
bar([1 2],[mean(data1) mean(data2)])
hold on
```

We also want to see the standard deviations using the function errorbar. This function as four inputs, the X location, the Y location, the size of the errorbars (i.e. the standard deviation) and the plotting style.

```
errorbar([1 2],[mean(data1) mean(data2)],[std(data1) std(data2)],'k.')
```

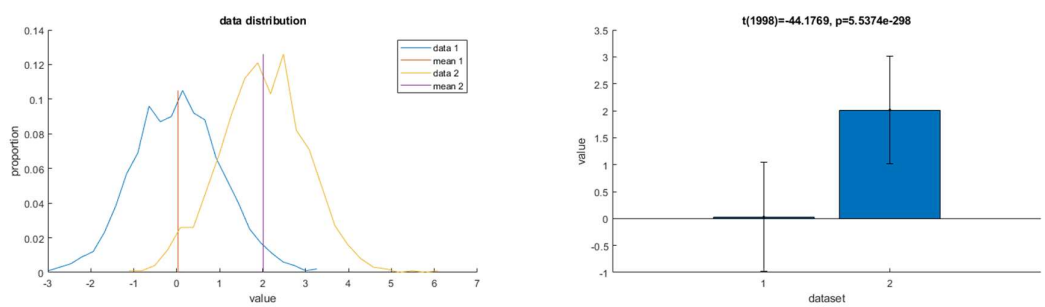
3. Test if the two distributions are significantly different using a two-sample t-test. This is similar to the one-sample t-test used in part 2 but it compares two distributions against each other, instead of one distribution against a single value.

```
%statistical test
[H,p,CI,stats]=ttest2(data,data2)
test=['t(' num2str(STATS.df) ')=' num2str(STATS.tstat) ' ,p=' num2str(P)]
```

You now have a script to compare two distributions of data. Save you script with the title 'FakeData' (we will use this script again in the 4th session).
My script and figure look like this:

```
clear all, close all, clc
% make some data
data1=randn(1,1000);
data2=randn(1,1000)+1;

% plot a histogram in the first subplot
subplot(2,2,1)
%data 1
[n,xout]=hist(data1,25);
plot(xout,n/sum(n)), hold on
plot([mean(data1) mean(data1)],[0 max(n/sum(n))])
%data 2
[n,xout]=hist(data2,25);
plot(xout,n/sum(n)), hold on
plot([mean(data1) mean(data1)],[0 max(n/sum(n))])

% anotation
title('data distribution')
xlabel('value')
ylabel('proportion')
legend([{'data'},{'mean'}])
box off % switch off the box arround the figure
set(gcf,'color','w') % set the color of the figure to white

% plot a bar plot of means in the second subplot
subplot(2,2,2)
bar([1 2],[mean(data1) mean(data2)])
hold on
errorbar([1 2],[mean(data1) mean(data2)],[std(data1) std(data2)],'k.')
box off % switch off the box arround the figure

%statistical test
[H,P,CI,STATS]=ttest2(data1,data2)
test=['t(' num2str(STATS.df) ')=' num2str(STATS.tstat) ' ,p=' num2str(P)]
title(test)
xlabel('dataset')
ylabel('value')
```

][



Part 3: look at the rod-balancing data

You now know how to

- Plot and summarize data (descriptive statistics).
- Statistically test if a distribution is different from an expected value (inferential statistics).
- Statistically test whether two distributions are different from each other (inferential statistics).

These are the most important parts of data analysis. In the rod balancing experiment we want to test whether:

- Speaking interferes with (i.e. slows down) the right hand more than the left hand.
- Whether this depend on sex and handedness

There are 5 steps to take:

1) Read and check the data.
2) Calculate mean balancing times per condition.
3) Calculate speaking interference.
4) Sort by handedness and sex and test for a difference between groups

Open a new script to write the analysis

1. Read and check the data: The data are saved in an excel sheet. Read the data with the function xlsread. You need to know the exact filename of the excel file (including the folder).

```
clear all, close all, clc
filename=['C:\Users\mark\OneDrive\teaching\PRA1005\2025\Course Material\Session 2\group_data_2025'];
[NUM,TXT,RAW]=xlsread(filename);
```

The complete sheet (text and numbers) is stored in RAW as a cell array. Any text in the sheet (the column titles) is stored in TXT and any numbers are stored in NUM. Each participant is represented in one row. Sex is coded in the first column with 1 indicating M and 2 indicating F and 3 indicating O. Handedness is coded in the second column with 1 indicating right-handed and 2 indicating left-handed. Use the functions length and find to count how many male, female, third gender/other and left- and right-handed and unknown participants there were:

```
N_participants=size(NUM,1)
N_male=length(find(NUM(:,1)==1))
N_female=length(find(NUM(:,1)==2))
N_other=length(find(NUM(:,1)==3))

N_RH=length(find(NUM(:,2)==1))
N_LH=length(find(NUM(:,2)==2))
N_OH=length(find(NUM(:,2)==0))
```

(the highlighted yellow warns you that the values will be displayed in the command window, you often do not want do this since it takes time, but here we want to see the results displayed).

2. Calculate mean balancing times per condition. Each participant completed each condition six times in random order. The data have been organized according to the column headings. Calculate the means per condition using the function mean. This function calculates the mean over columns (i.e. participants) by default, but we want to calculate over rows (i.e. first to sixth repetition). The second input to mean tells it which dimension to use – dimension 2 in our case:

```
Mean_times(:,1)=mean(NUM(:,3:8),2); % right silent
Mean_times(:,2)=mean(NUM(:,9:14),2); % right speaking
Mean_times(:,3)=mean(NUM(:,15:20),2); % left silent
Mean_times(:,4)=mean(NUM(:,21:26),2); % left speaking
```

3. There is a lot of variation in how long people can balance for. Calculate the mean balancing time over all conditions and normalize the per-condition-means by this number.

```
Overall_times=mean(NUM(:,3:26),2); %the average over all trys
Mean_times=Mean_times./Overall_times; %normalize
%notice that you use ./ for element-wise division
```

4. Calculate interference between speaking and silence for the left and right hand.

```
interference(:,1)=(Mean_times(:,2)-Mean_times(:,1));%Right hand
interference(:,2)=(Mean_times(:,4)-Mean_times(:,3));%Left hand
```

Calculate the difference in interference for the right and left hand

```
interference_difference=interference(:,1)-interference(:,2);%right-left
```

You can now use the interference difference data as you used data1 in the practice analysis. Copy and paste the lines from your script fake_data.m that performs the descriptive statistics for one sample of data.

```
% plot a histogram
subplot(2,2,1)
[n,xout]=hist(Interference_difference,25);% use Interference_difference instead of data1
plot(xout,n/sum(n)), hold on
plot([mean(Interference_difference) mean(Interference_difference)],[0 max(n/sum(n))])

% anotation
title('full data')
xlabel('Interference difference')
ylabel('proportion')
legend([{'data'},{'mean'}])
box off % switch off the box arround the figure
set(gcf,'color','w') % set the color of the figure to white
```

Now perform a significance test. The null hypothesis is that there is no difference in interference for the left and right hand. In other words we expect the difference between interference for the left and right hand to be equal to 0. This effectively calculates a paired t-test.

```
%statistical test
[h,p,ci,stats]=ttest(interference_difference,0);
test=['t= (' num2str(stats.df) ') = ' num2str(stats.tstat) ' p = ' num2str(p)];
text(xout(1),max(n/sum(n)),test)
```

Between groups comparisons

5. Right-handed people are thought to have more strongly lateralized brains than left-handed people. Repeat separately for left and right-handed participants using find to separate them.

```
subplot(2,2,2)
Right_handed=interference_difference(find(NUM(:,2)==1),:);
Left_handed=interference_difference(find(NUM(:,2)==2),:);

bar([1 2],[mean(Right_handed) mean(Left_handed)]); hold all
errorbar([1 2],[mean(Right_handed) mean(Left_handed)],[std(Right_handed) std(Left_handed)],'k.');
box off
set(gca,'xticklabels',[{'right'},{'left'}])
```

Perform a statistical test to see if the amount of interference is the same in both groups. You could also test if the effect is present in both groups separately.

```
%statistical test
[H,P,CI,STATS]=ttest2(Left_handed,Right_handed)
test=['t(' num2str(STATS.df) ')=' num2str(STATS.tstat) ' ,p=' num2str(P)]
title(test)
```

It is interesting to check whether right-handed people perform better with their right hands than their left, and whether the reverse is true for left-handed people. See if you can perform this analysis without the worksheet!

6. For reasons unknown, males tend to be better at rod balancing than females (Porras et al., 2015). Check if this is the case in our data.

```
male=Overall_times(find(NUM(:,1)==1));
female=Overall_times(find(NUM(:,1)==2));

subplot(2,2,3)
[~,xout]=hist(Overall_times,25); % find good bins for the data
[n,xout]=hist(male,xout);
plot(xout,n/sum(n),'linewidth',2,'color','r'); hold all
[n,xout]=hist(female,xout);
plot(xout,n/sum(n),'linewidth',2,'color','b')

plot([mean(male) mean(male)],[0 max(n/sum(n))],'r:')
plot([mean(female) mean(female)],[0 max(n/sum(n))],'b:')

title('data'); % you might use a different title
xlabel('values')
ylabel('proportion')
legend([{'data1'},{'data2'}])
box off
set(gcf,'color','w')

[h,p,ci,stats]=ttest2(male,female);
test=['t= (' num2str(stats.df) ') = ' num2str(stats.tstat) ' p = ' num2str(p)];
text(xout(1),max(n/sum(n)),test)
```

7. It might also be true that there is a difference in the amount of language lateralization between males and females (Hiscock et al., 2001). Use what you have learned to test this and display the results in another subplot.

References

Hiscock, M., Perachio, N., & Inch, R. (2001). Is there a sex difference in human laterality? IV. An exhaustive survey of dual-task interference studies from six neuropsychology journals.

Journal of Clinical and Experimental Neuropsychology, 23(2), 137–148. https://doi.org/10.1076/jcen.23.2.137.1206
Porras, I. C., Hiscock, M., & Layne, C. S. (2015). The sex difference in rod balancing: confirmation of the difference and a test of three hypothetical explanations. Perceptual and Motor Skills, 121(3), 706–726. https://doi.org/10.2466/22.23.PMS.121c25x7

How does this compare to other methods in neuroscience?

In neuroscience we are interested in the behaviour of neurons, but often we only have indirect measures of neuronal activity. In fMRI research we use measures of blood oxygenation, in MEG or EEG research we use measures of the magnetic or electrical fields produced by synchronized neuronal activity, just as in the rod balancing experiment, we used balancing time as an indirect measure. Because the brain is always active, simply measuring activity (especially with indirect measures) does not tell you much about what the brain is doing or what the activity represents. Instead we look at the difference in activity between two conditions (e.g. speaking and not speaking). When you see results showing that a particular area 'lights up' during a particular function, what is being represented is the difference between signal recorded while that function is being performed and the activity recorded during a control condition (no function or a different function being performed).

The principal of investigating neuronal and mental processes by measuring time differences (known as 'mental chronometry') was developed by the 19th century Dutch scientist Franciscus Cornelius Donders, after whom the *Donders Institute for Brain, Cognition and Behaviour* (Radboud University, Nijmegen) is named.

Opening of the Centre for Cognitive Neuroimaging (2002). In the presence of various officials and representatives of the University, Her Majesty Queen Beatrix unveils a bronze bust of *F.C. Donders*, the namesake of the Donders Institute and father of mental chronometry.